

---

## **TP3 BDA: Votre premier micro-service, Hibernate, Base de données in memory (H2)**

---

ING2 INFORMATIQUE  
Sup Galilée

**Réalisé par**  
Daniel THARMARAJAH 12000230

# Contents

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Introduction</b>                                    | <b>3</b>  |
| <b>Prérequis</b>                                       | <b>3</b>  |
| <b>Création de projet</b>                              | <b>3</b>  |
| <b>Lancer l'application</b>                            | <b>6</b>  |
| <b>Accès à la page</b>                                 | <b>7</b>  |
| <b>Écriture de services REST</b>                       | <b>7</b>  |
| Créons notre premier service REST . . . . .            | 8         |
| Modification des propriétés de l'application . . . . . | 8         |
| Accès à des objets . . . . .                           | 9         |
| Tester l'API REST . . . . .                            | 10        |
| Passer des paramètres dans une API REST . . . . .      | 12        |
| Accéder à une collection . . . . .                     | 13        |
| Méthodes HTTP dans une API REST . . . . .              | 14        |
| POST . . . . .                                         | 15        |
| DELETE . . . . .                                       | 15        |
| PUT . . . . .                                          | 17        |
| <b>Base de Données</b>                                 | <b>18</b> |
| H2 . . . . .                                           | 18        |
| Création de packages . . . . .                         | 18        |
| MySQL . . . . .                                        | 22        |

# Introduction

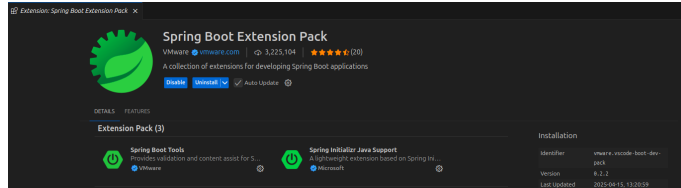
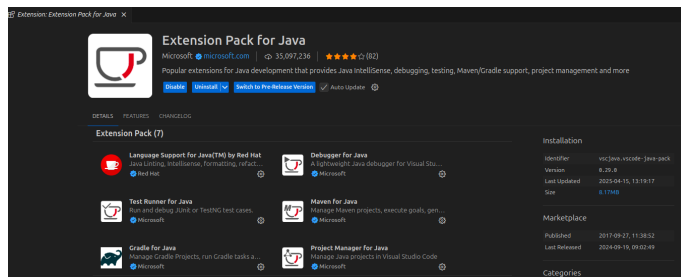
Pour ce TP, j'ai opté pour l'éditeur de texte Visual Studio Code pour programmer en Spring et donc j'ai décidé d'en faire un sorte de tutoriel.

## Prérequis

Voici les prérequis nécessaires :

- **Java** : Version 17, 21 ou 23
- **XAMPP** : MySQL

Installer les extension suivantes sur VSCode

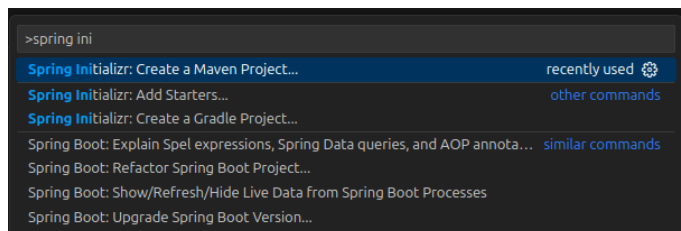


**Attention** : La version de Java utilisée est la version 21

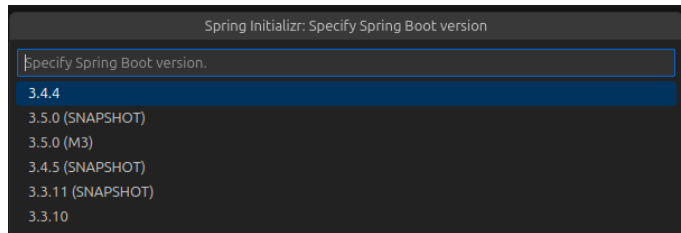
## Création de projet

Suivez ces étapes pour créer un projet Spring avec Visual Studio Code :

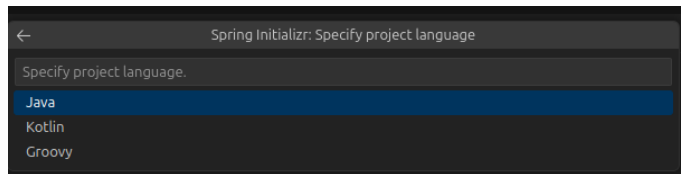
1. Faites **Ctrl + Shift + P**.
2. Tapez **Spring Initializr** et sélectionnez **Spring Initializr : Create a Maven Project**.



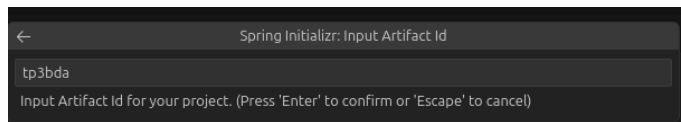
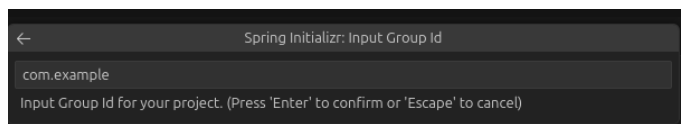
3. Choisissez une version de Spring.



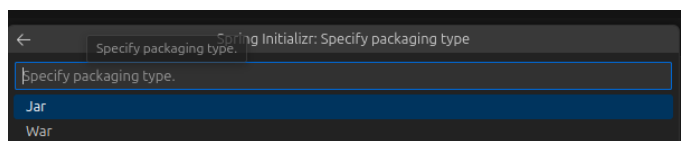
4. Choisissez **Java** comme langage.



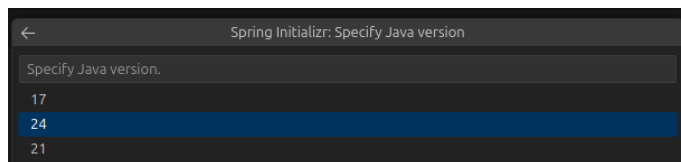
5. Entrez un **GroupId** et un **ArtifactId** au hasard.



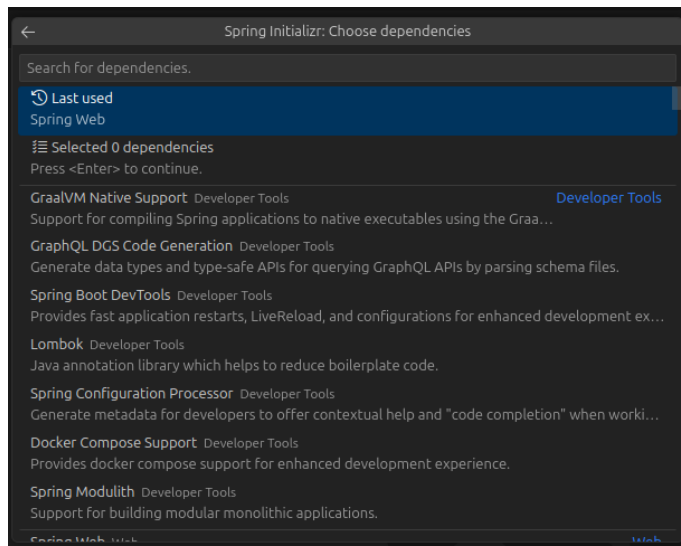
6. Sélectionnez **Jar**.



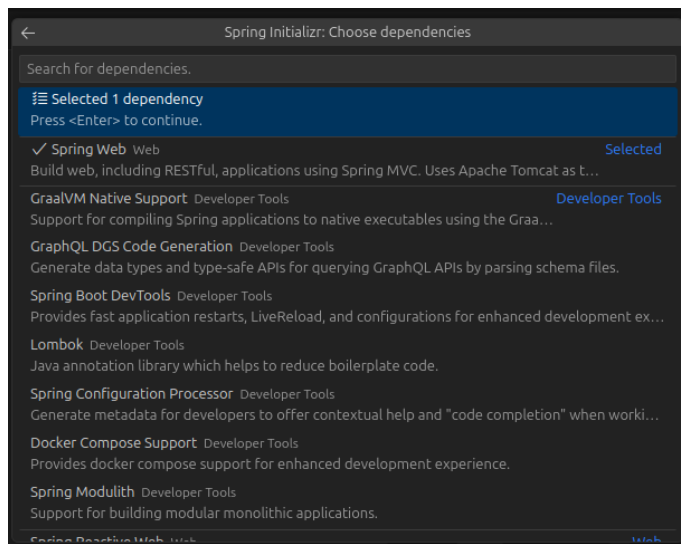
7. Sélectionnez une des versions proposées.

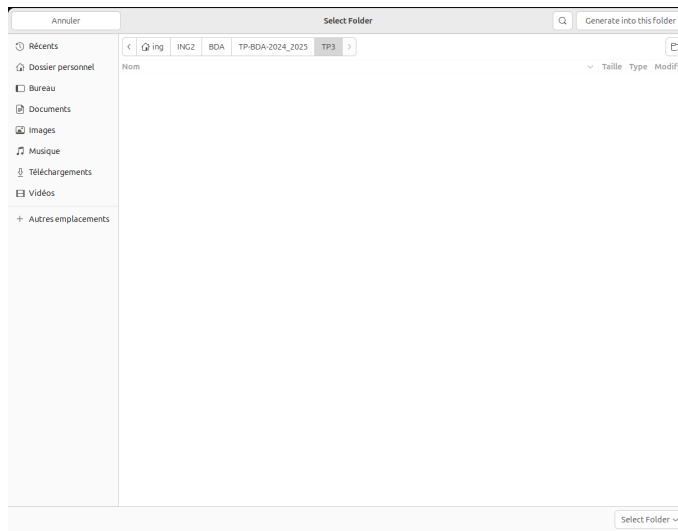


8. Tapez **Spring Web** et sélectionnez cette dépendance.

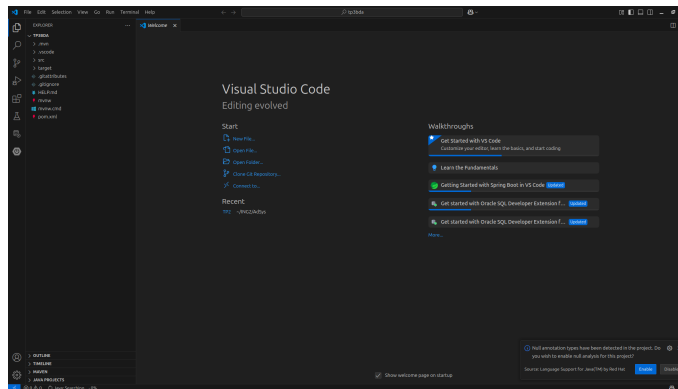


9. Appuyez sur **Entrée** et sélectionnez le répertoire où vous voulez que ce projet soit situé.





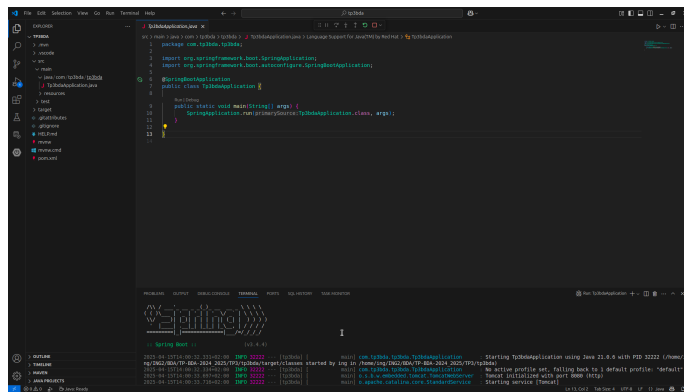
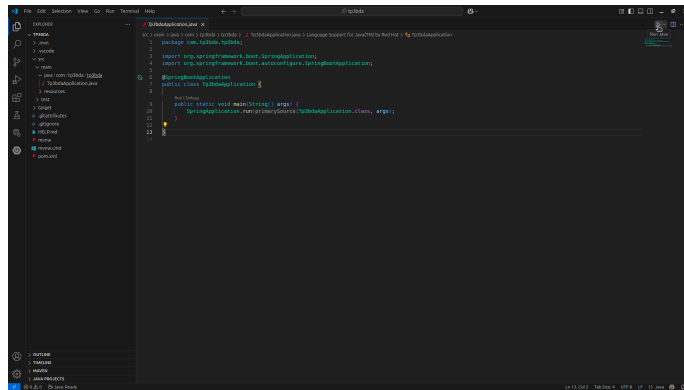
10. Vous pouvez à présent ouvrir le projet.



NB : Si vous avez un problème avec la version de java, vous pouvez modifier le fichier pom.xml du projet et les propriétés concernant la version de java.

## Lancer l'application

Pour lancer votre application, parcourez le répertoire `src/main` et cherchez un fichier ayant comme nom `<nom_projet>Application`. Appuyez sur le petit bouton en haut à droite pour lancer l'application.



Voici le résultat attendu :

## Accès à la page

Par défaut, la page se trouve sur le port 8080 à cette adresse : <http://localhost:8080/>



## Écriture de services REST

**NB :** Toutes les classes, applications ou services REST que vous écrivez doivent être situés dans le même répertoire que l'application principale.

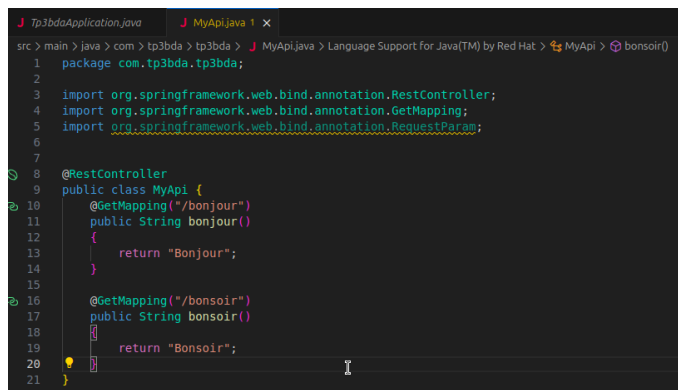
## Créons notre premier service REST

- Créez un nouveau fichier Java dans le même répertoire que l'application principale.
- Au-dessus de la déclaration de la classe, ajoutez l'annotation `@RestController`.
- Définissez les méthodes nécessaires pour ce service.

Dans notre cas, nous allons afficher **Bonjour** et **Bonsoir**. Pour cela, créez deux méthodes qui renvoient chacune une chaîne de caractères, respectivement "Bonjour" et "Bonsoir", et annotez-les avec `@GetMapping`.

Cette annotation permet de définir des points d'entrée HTTP de type GET. Par exemple, si vous mettez `/bonjour`, vous accéderez à cette méthode via : `http://localhost:8080/bonjour`

Voici le code correspondant :

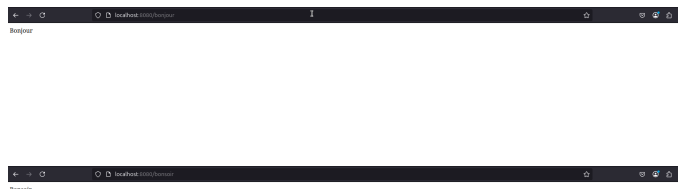


```
src > main > java > com > tp3bda > tp3bda > J MyApi.java > Language Support for Java(TM) by Red Hat > MyApi > bonsoir()
1 package com.tp3bda.tp3bda;
2
3 import org.springframework.web.bind.annotation.RestController;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestParam;
6
7
8 @RestController
9 public class MyApi {
10     @GetMapping("/bonjour")
11     public String bonjour()
12     {
13         return "Bonjour";
14     }
15
16     @GetMapping("/bonsoir")
17     public String bonsoir()
18     {
19         return "Bonsoir";
20     }
21 }
```

**Relancez l'application** et testez l'accès aux deux méthodes via les URLs suivantes :

- `http://localhost:8080/bonjour`
- `http://localhost:8080/bonsoir`

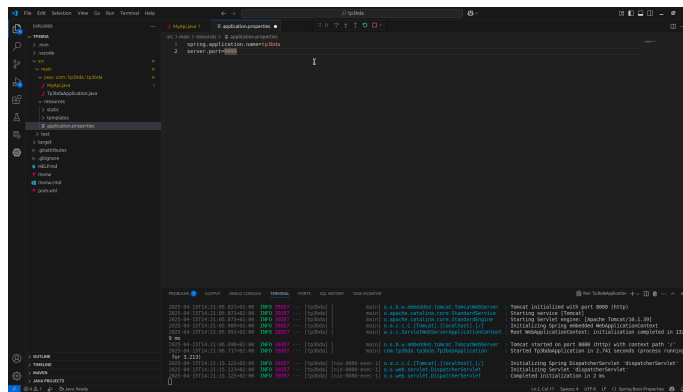
Voici le rendu :



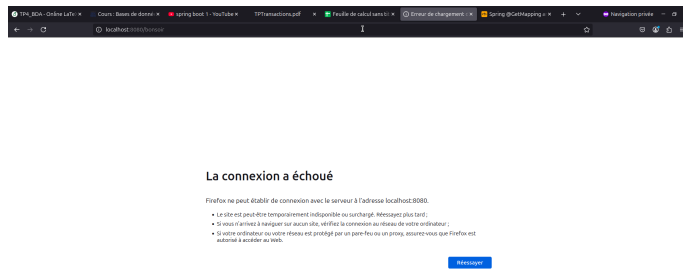
## Modification des propriétés de l'application

Il est possible de modifier les propriétés de votre application, notamment en modifiant le fichier `src/main/resources/application.properties`.

Par exemple, voici comment modifier le numéro de port :



On relance l'application Si on tente d'accéder à `http://localhost:8080/bonsoir`, voici le résultat :

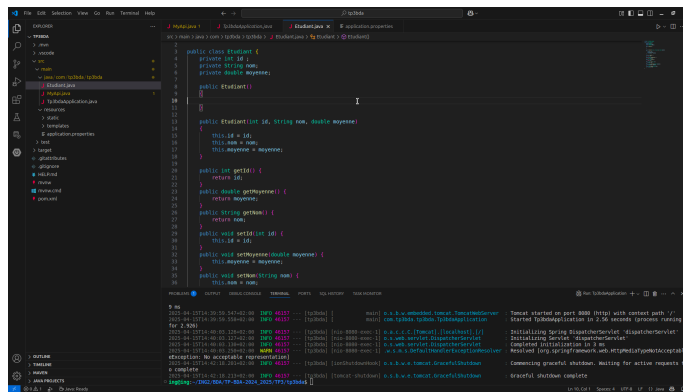


Par contre `http://localhost:9999/bonsoir` donnera :

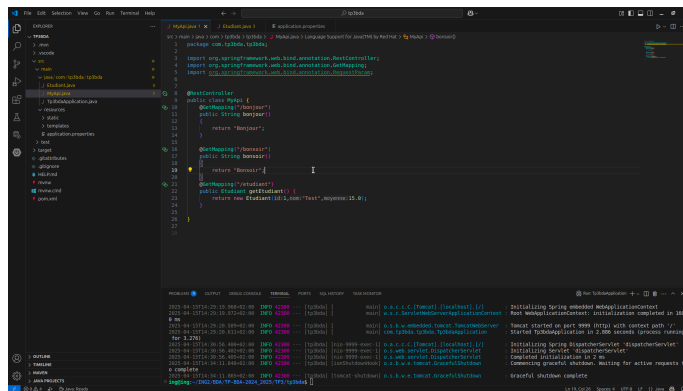


## Accès à des objets

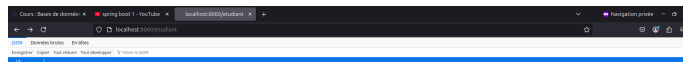
On peut également accéder à des objets. Voici un exemple avec la classe `Etudiant`. On définit la classe `Étudiant` de la manière suivante : **N'oubliez pas d'implémenter les getters et les setters.**



Pour tester, on va créer la méthode suivante dans la classe MyApi :



Voici le rendu après lancemenet de l'application et test de la méthode définie avant :



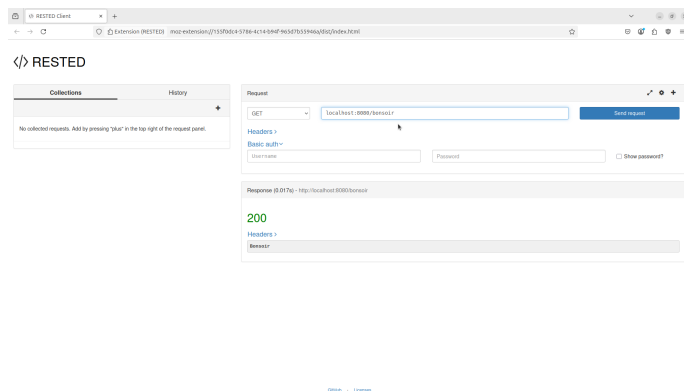
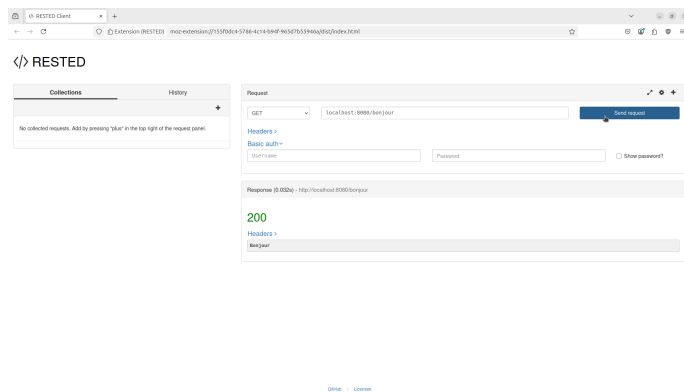
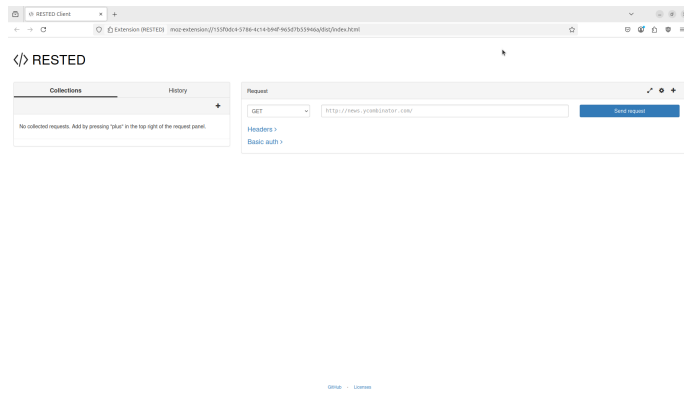
## Tester l'API REST

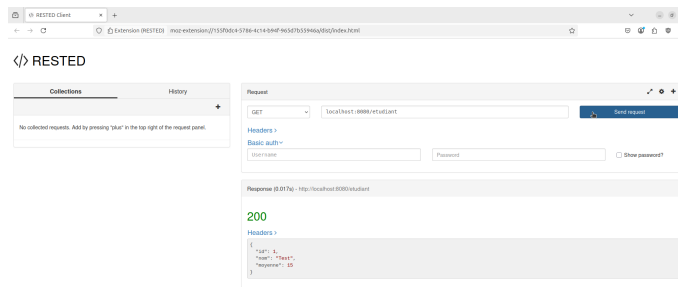
Pour cette partie, j'ai opté pour l'extension **Firefox RESTED**.

Avec cette extension, vous pouvez facilement tester vos méthodes API REST implémentées précédemment. Vous pouvez ainsi retester les méthodes que nous avons créées auparavant :

- `http://localhost:8080/bonjour` pour la méthode `bonjour()`
- `http://localhost:8080/bonsoir` pour la méthode `bonsoir()`

Dans RESTED, vous pouvez spécifier l'URL de l'API, sélectionner la méthode **GET**, et envoyer la requête pour voir la réponse.



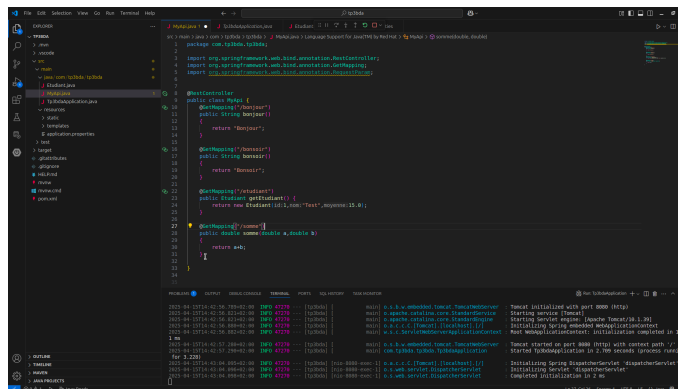


## Passer des paramètres dans une API REST

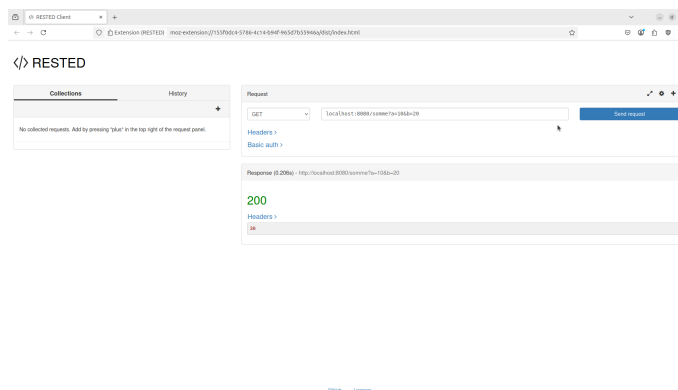
Voyons maintenant comment passer des paramètres dans une API REST. Voici un exemple avec la somme de deux entiers.

On va créer une méthode qui accepte deux paramètres entiers, les additionne et renvoie le résultat.

Dans la classe `MyApi`, nous ajoutons la méthode suivante :



Voici l'URL utilisée : `http://localhost:8080/somme?a=10&b=20` et son test



Cela est aussi valable sur navigateur

**Attention veuillez a bien passer les paramètres dans l'URL.**



## Accéder à une collection

Voyons maintenant comment accéder à une collection d'objets via une API REST.

Pour ce faire, nous allons déclarer une collection d'étudiants statique et l'initialiser dans la classe MyApi comme ceci

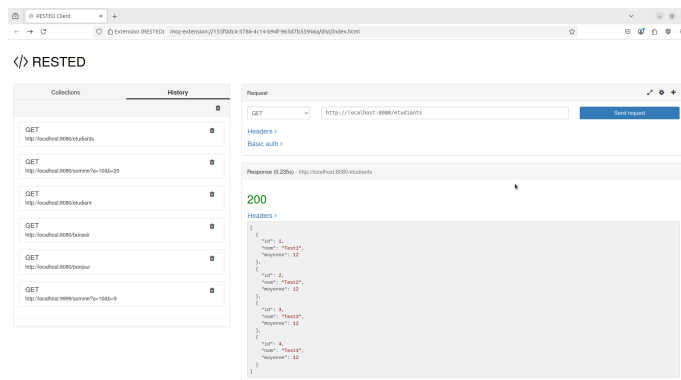
```
public class MyApi {
    public static ArrayList<Etudiant> listeEtu = new ArrayList<>();

    static {
        listeEtu.add(new Etudiant(id:1,nom:"Test1",moyenne:12));
        listeEtu.add(new Etudiant(id:2,nom:"Test2",moyenne:12));
        listeEtu.add(new Etudiant(id:3,nom:"Test3",moyenne:12));
        listeEtu.add(new Etudiant(id:4,nom:"Test4",moyenne:12));
    }
}
```

On va aussi définir une méthode pour accéder à cette collection :

```
@GetMapping("/etudiants")
public Collection<Etudiant> getAllEtudiants() {
    return listeEtu;
}
```

Voici le résultat avec RESTED :



Ce résultat est aussi valable sur navigateur :

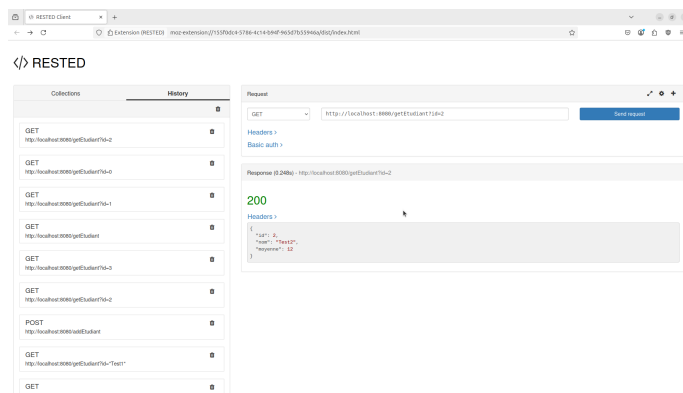


On peut aussi accéder à un élément d'une collection, Voici un exemple avec la liste d'étudiants pour récupérer un étudiant via son id

On définit d'abord une méthode pour accéder un élément de la collection :

```
@GetMapping("/getEtudiant")
public Etudiant getEtudiant(int id) {
    for(Etudiant e : listeEtu)
    {
        if(e.getId() == id)
        {
            return e;
        }
    }
    return null;
}
```

Voici le résultat avec l'adresse : localhost:8080/getEtudiant?id=2



## Méthodes HTTP dans une API REST

Jusqu'à maintenant, nous nous contentons d'implémenter uniquement des requêtes GET. Cependant, il est possible d'implémenter d'autres types de requêtes HTTP.

Avant tout, voici un rappel sur les méthodes HTTP courantes et les annotations associées dans Spring Boot :

- **GET** : Pour recevoir une ressource.  
Utilisez l'annotation **@GetMapping**.

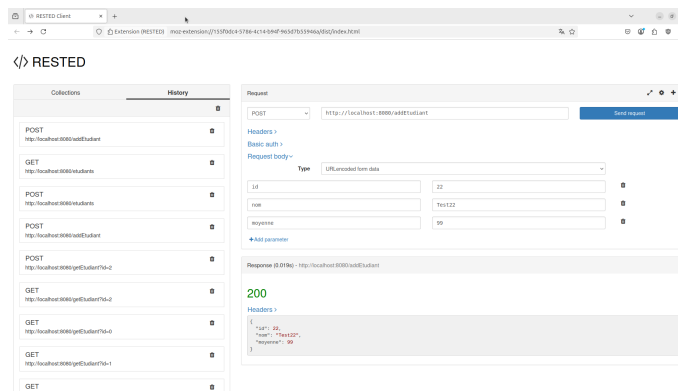
- **POST** : Pour créer une nouvelle ressource.  
Utilisez l'annotation `@PostMapping`.
- **PUT** : Pour modifier une ressource existante.  
Utilisez l'annotation `@PutMapping`.
- **DELETE** : Pour supprimer une ressource.  
Utilisez l'annotation `@DeleteMapping`.

## POST

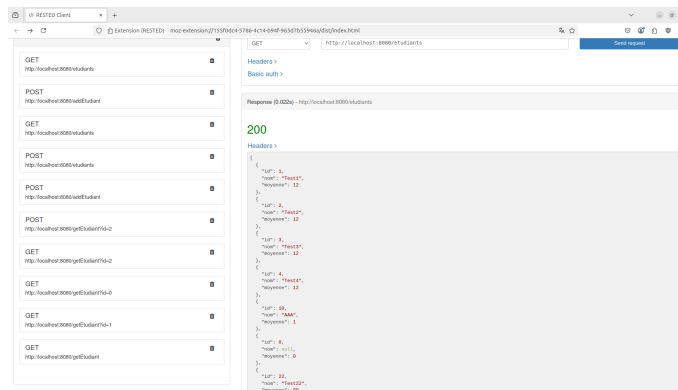
Voici la méthode implémentée et son résultat :

```
@PostMapping("/addEtudiant")
public Etudiant addEtudiant(Etudiant etu) {
    listeEtu.add(etu);
    return etu;
}
```

Attention : Pour passer les paramètres avec RESTED, il faut spécifier comme type de corps de requête : **URLencoded form data**



On vérifie que cet élément à bien été ajoutée :



## DELETE

Voici la méthode implémentée :

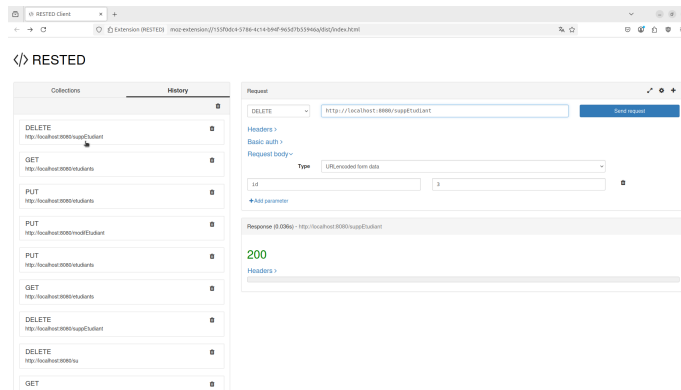
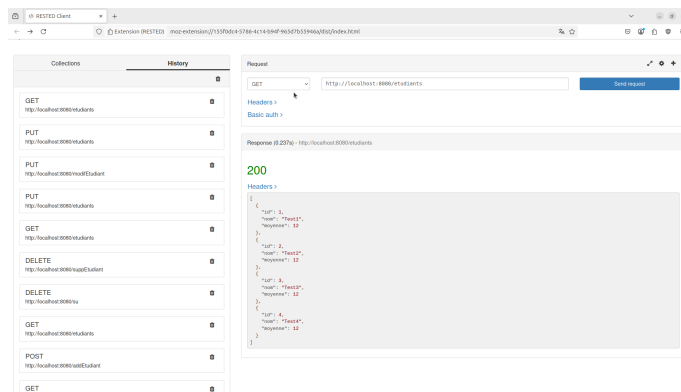
```

@DeleteMapping("/suppEtudiant")
public void suppEtudiant(int id) {
    Iterator<Etudiant> i = listeEtu.iterator();

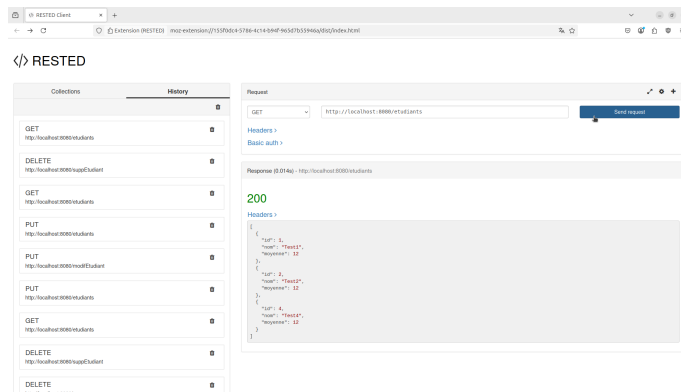
    while(i.hasNext()) {
        Etudiant e = i.next();
        if (e.getId() == id) {
            i.remove();
        }
    }
}

```

Voici la liste avant suppression :

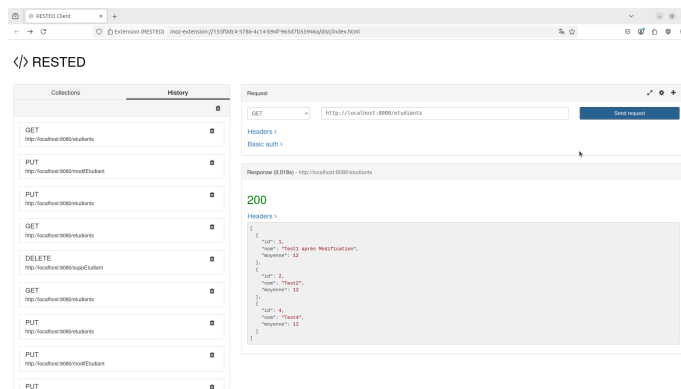
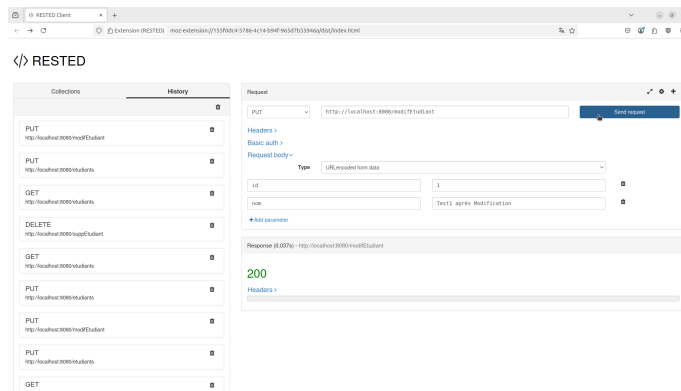
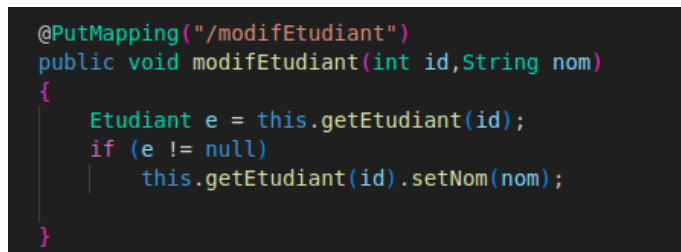


Voici la liste après suppression :



## PUT

Voici la méthode implémentée et son résultat :

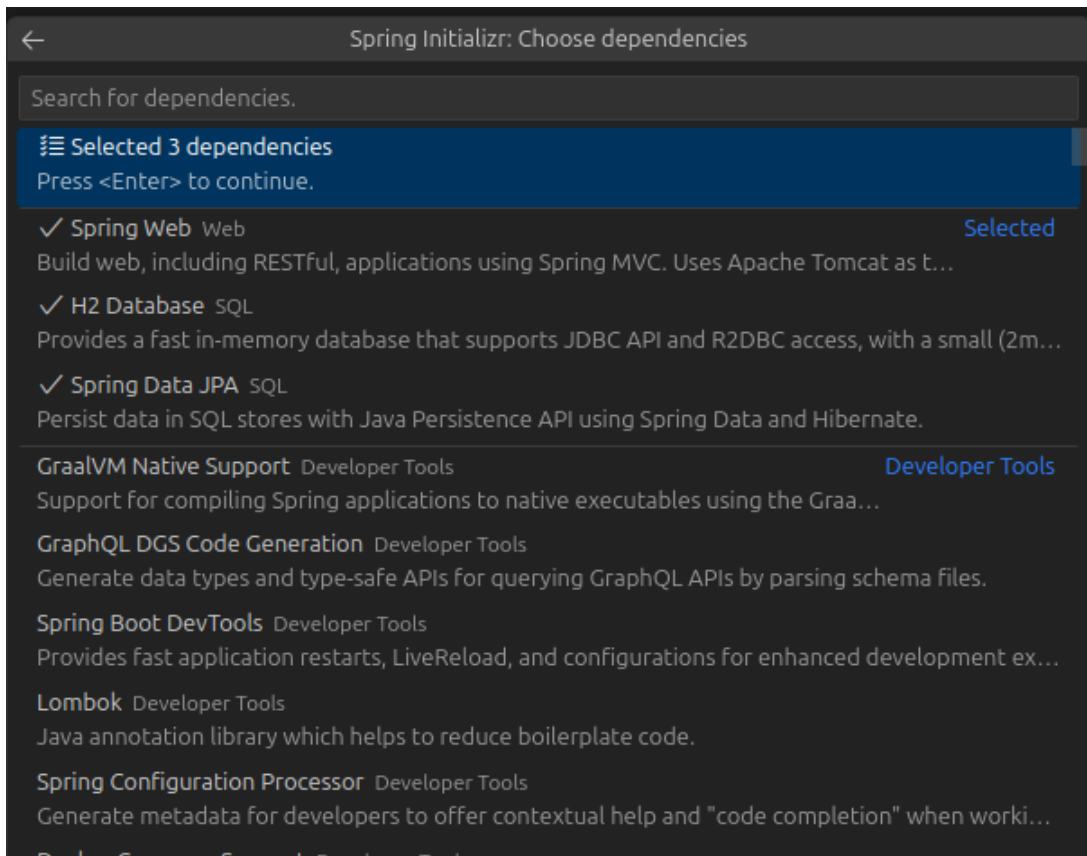


## Base de Données

Nous allons maintenant manipuler des bases de données avec Spring Boot en utilisant les annotations JPA.

Avant de commencer cette partie, nous allons créer un nouveau projet avec les dépendances suivantes :

- **Spring Web** : Pour créer des applications web avec Spring Boot.
- **H2 Database** : Pour utiliser une base de données en mémoire H2.
- **Spring Data JPA** : Pour gérer l'accès aux données via Java Persistence API (JPA).

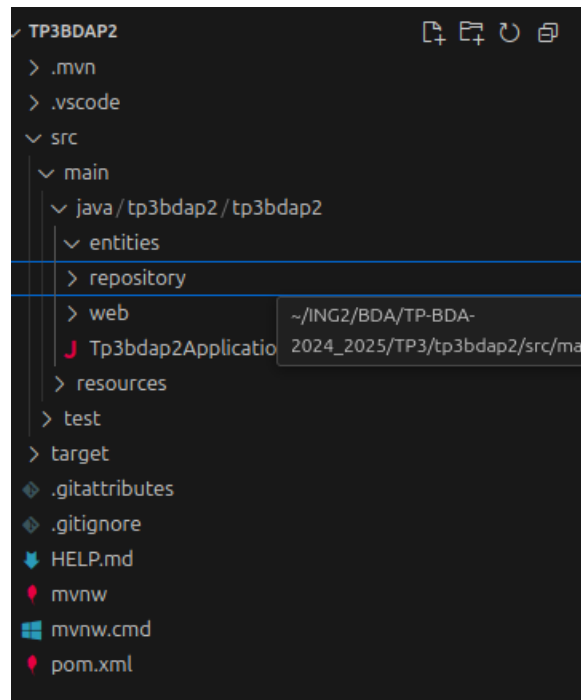


## H2

On va tout d'abord manipuler une base de données H2, une base de données en mémoire.

### Création de packages

Après avoir recréé un projet Spring avec les dépendances mentionnées précédemment, nous allons créer trois packages sous le dossier `src/main/java`, dans le même répertoire que l'application principale :



- **web** : pour les contrôleurs (controllers),
- **entities** : pour les classes représentant les entités,
- **repository** : pour les interfaces permettant d'interagir avec la base de données.

Nous allons maintenant créer une entité nommée Adhérent comme ceci en définissant aussi les getters et les setters :

```
public class Adherent {

    private Long id;
    private String nom;
    private String ville;
    private int age;
```

Nous allons maintenant définir cette classe comme une entité à l'aide des annotations JPA :

- **@Entity** : à placer au-dessus de la classe pour indiquer qu'il s'agit d'une entité,
- **@Id** : pour désigner l'attribut correspondant à la clé primaire,
- **@GeneratedValue(strategy = GenerationType.AUTO)** : pour que la valeur de la clé primaire soit générée automatiquement.

```

@Entity
public class Adherent {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String nom;
    private String ville;
    private int age;
    /* Définir aussi les getters et setters */
}

```

Maintenant on va définir une interface dans le package repository pour une interface pour cette classe avec comme paramètre la classe et le type de la clé primaire comme ceci :

```

package tp3bdap2.tp3bdap2.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import tp3bdap2.tp3bdap2.entities.Adherent;

public interface AdherentRepository extends JpaRepository<Adherent, Long> {

}

```

**NB :** Cette interface hérite de l'interface JpaRepository et ne pas oublier l'annotation @Repository au-dessus cette interface

Après avoir défini l'interface, on va définir la méthode suivante dans la classe principale. On mettra l'annotation @Bean de spring pour executer automatiquement la méthode définie lors du lancement de l'application

```

@SpringBootApplication
public class Tp3bdap2Application {

    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(primarySource:Tp3bdap2Application.class, args);
    }

    @Bean
    CommandLineRunner runner(AdherentRepository repository)
    {
        return args -> {
            AdherentRepository repository = tp3bdap2.tp3bdap2.Tp3bdap2Application.runner(AdherentRepository
            repository.saveAndFlush(new Adherent(id:null, nom:"A", ville:"B", age:29));
            repository.saveAndFlush(new Adherent(id:null, nom:"C", ville:"D", age:30));
            repository.saveAndFlush(new Adherent(id:null, nom:"E", ville:"F", age:31));
            repository.saveAndFlush(new Adherent(id:null, nom:"G", ville:"H", age:32));
        };
    }
}

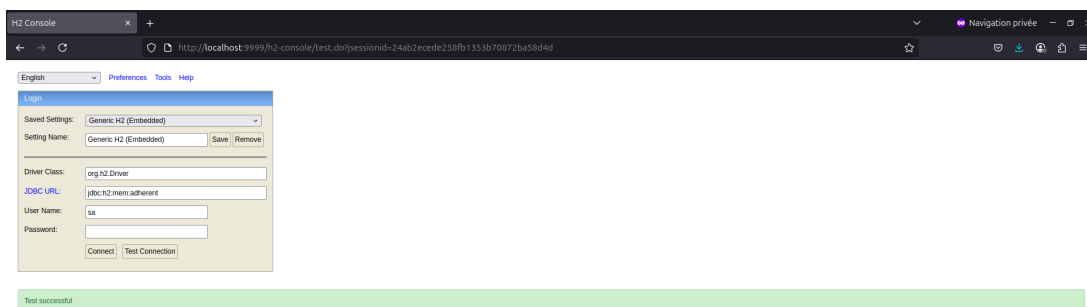
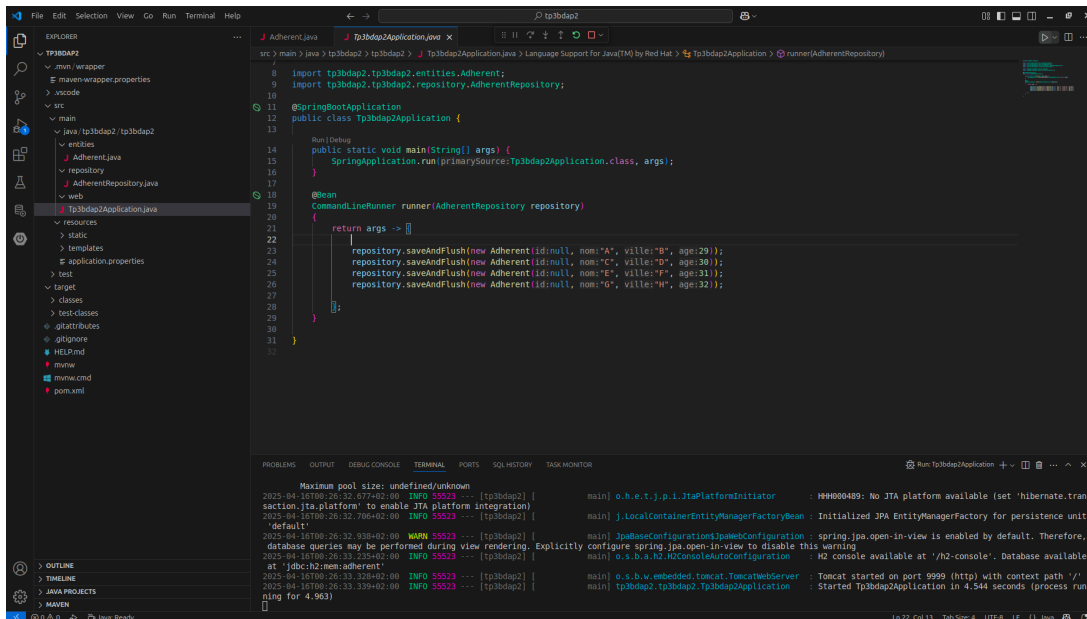
```

Avant de terminer, il faut configurer les propriétés de l'application comme ceci dans le fichier **application.properties** :

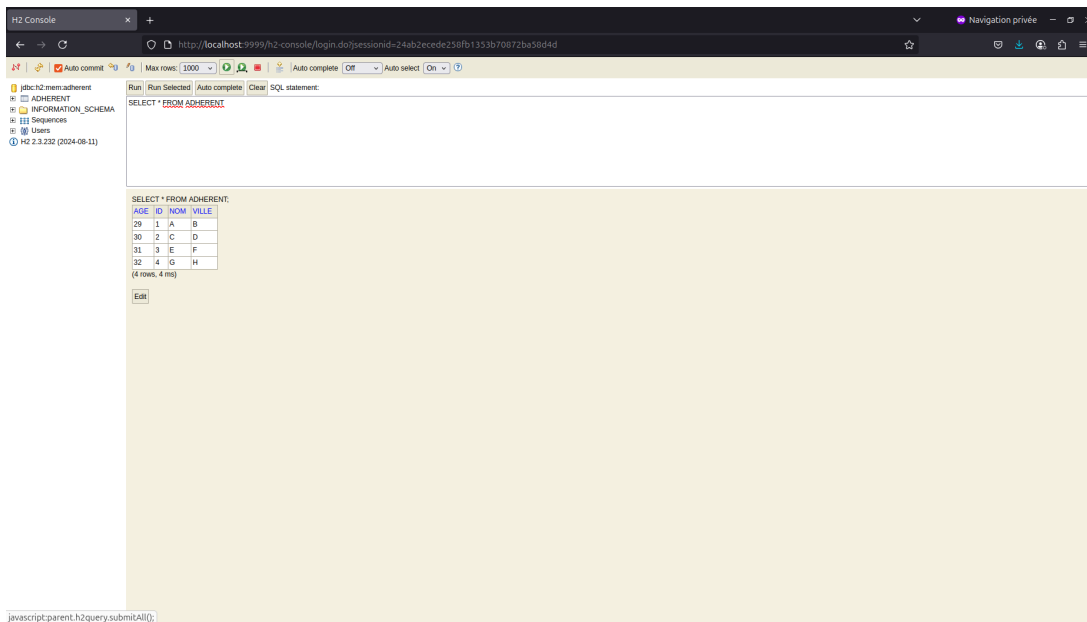
```
spring.application.name=tp3bdap2
server.port=9999
spring.datasource.url=jdbc:h2:mem:adherent
spring.h2.console.enabled=true
```

Maintenant on peut lancer l'application

On peut accéder à la console de H2 avec le lien suivant : <http://localhost:9999/h2-console>



Après être connecté à la console, on peut lister les adhérents ajoutés avec la méthode runner :



## MySQL

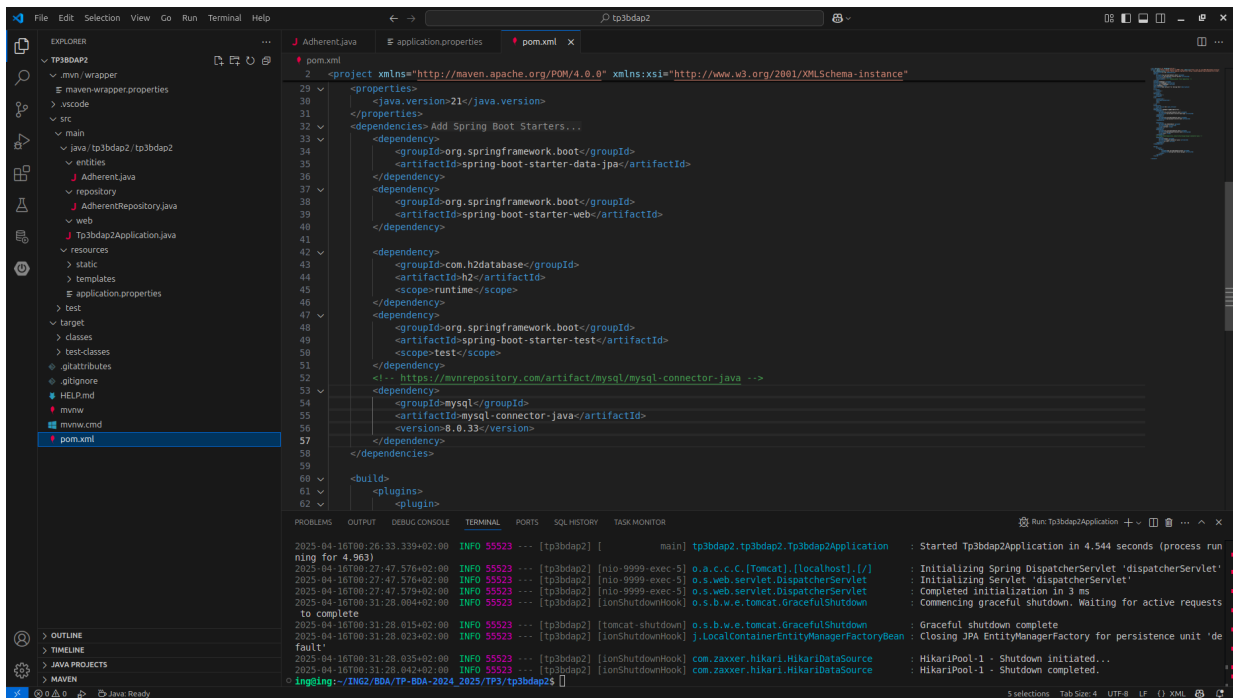
On va maintenant passer par une BDD MySQL. Tout d'abord, il faut modifier le fichier `application.properties` comme ceci :

```

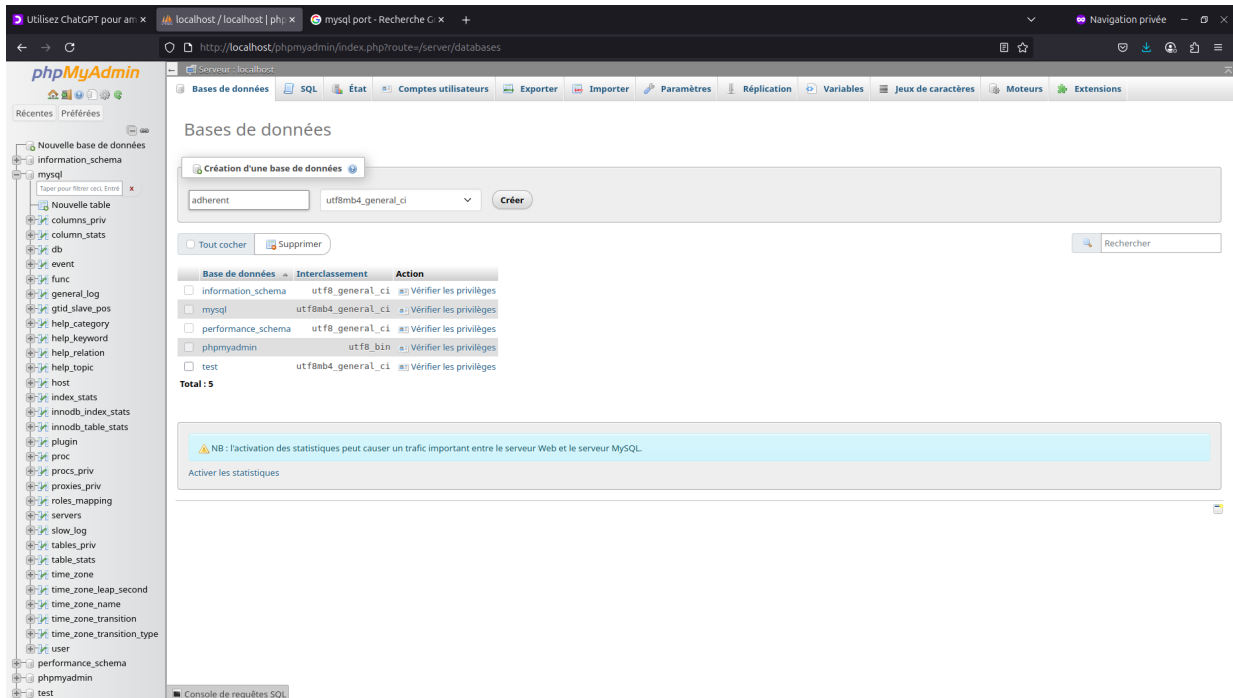
J Adherent.java  J Tp3bdap2Application.java  application.properties x  pom.xml
src > main > resources > application.properties
1  spring.application.name=tp3bdap2
2  server.port=9999
3  spring.datasource.url=jdbc:mysql://localhost:3306/adherent
4  spring.datasource.username=root
5  spring.jpa.hibernate.ddl-auto=create-drop
6
7

```

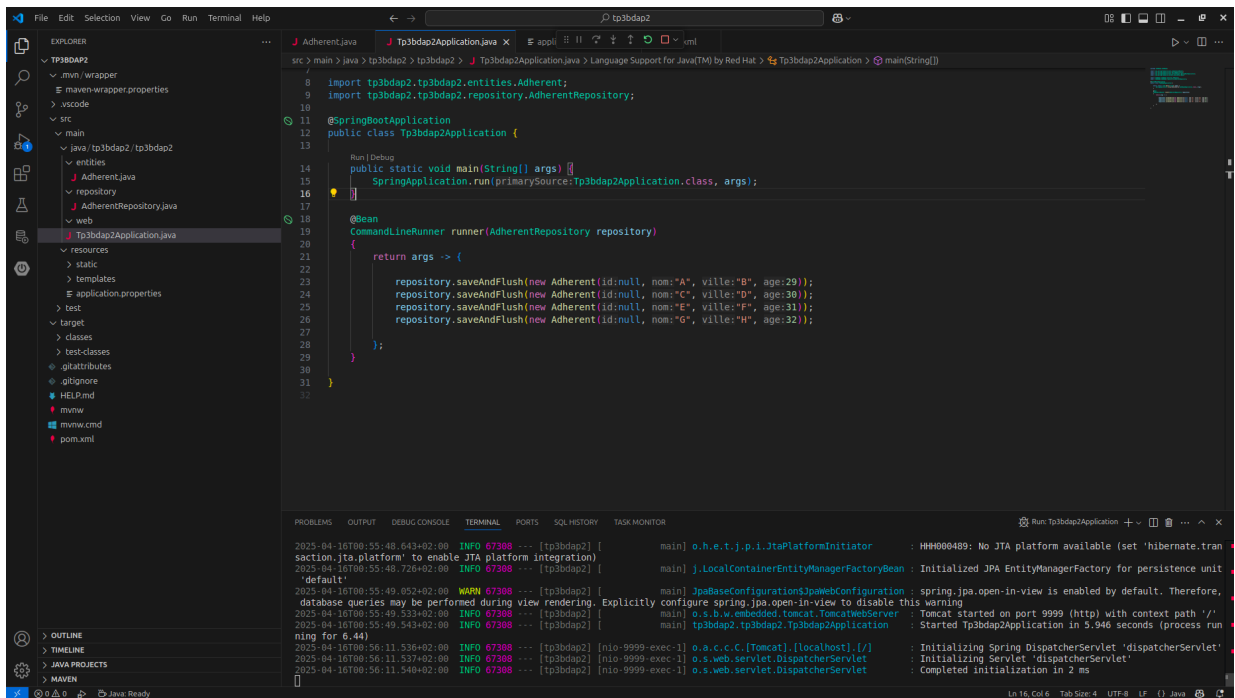
il faut aussi ajouter le driver MySQL a notre projet en ajoutant la dépendance suivante :



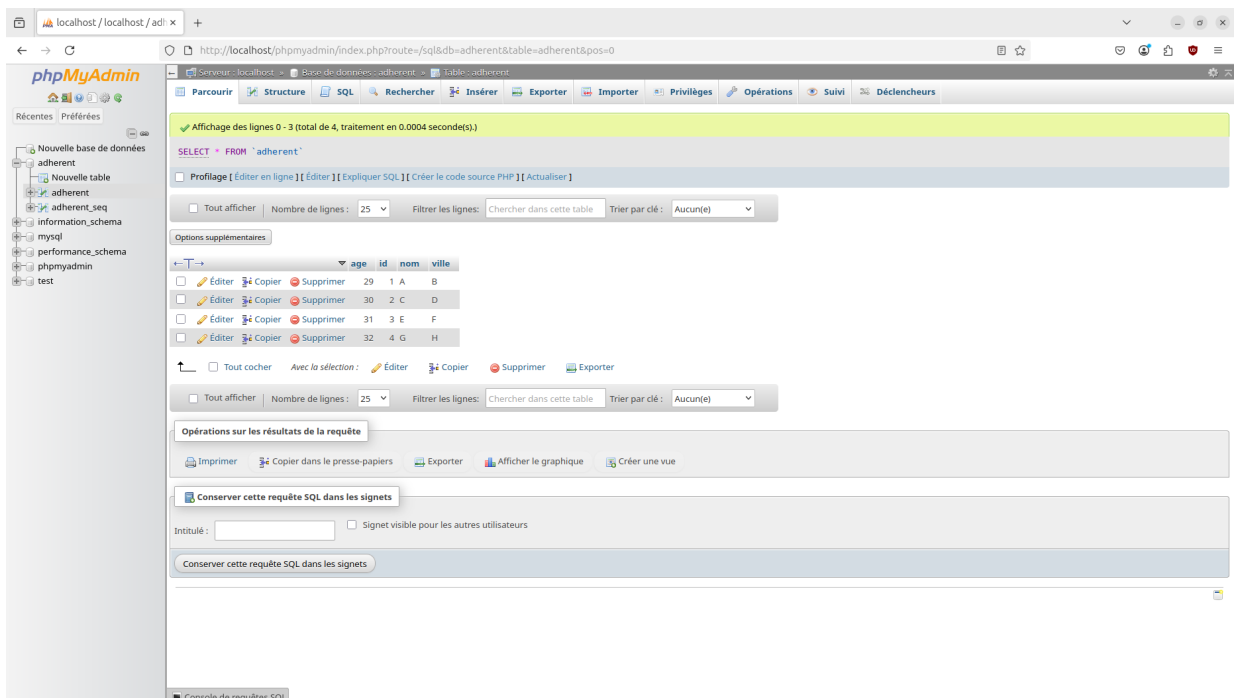
Puis dans PHPMyAdmin creer une base de donnees nommee adherent :



On lance maintenant l'application : NB: J'ai du relancer le serveur XAMPP pour que la connexion a la Base de données réussisse



On verifie le contenu de la table adherent :



Et c'est tout bon.